14. 서명 및 슬래싱 보호 구현 분석

개요

Lighthouse에서 검증자의 서명 과정과 슬래싱 보호는 Ethereum 2.0 합의 프로토콜의 핵심 보안 메커니즘입니다. 이 문서는 `signing_method` 와 `slashing_protection` 모듈의 실제 구현을 분석하여, 검증자가 어떻게 안전하게 메시지에 서명하고 슬래싱을 방지하는지 상세히 설명합니다.

서명과 슬래싱 보호는 다음과 같은 계층적 구조로 구현됩니다:

1. **서명 방법 (SigningMethod)** - 로컬 키스토어 또는 원격 서명자를 통한 서명
2. **슬래싱 보호 데이터베이스** - SQLite 기반의 슬래싱 방지 메커니즘
3. **통합 검증자 스토어** - 서명과 슬래싱 보호를 결합한 안전한 서명 인터페이스

서명 방법 (SigningMethod) 구현

SigningMethod 열거형 구조

```
// validator_client/signing_method/src/lib.rs

pub  enum SigningMethod {

/// EIP-2335 키스토어를 사용한 로컬 서명

LocalKeystore {

voting_keystore_path: PathBuf,

voting_keystore_lockfile: Mutex<Option<Lockfile>>,

voting_keystore: Keystore,

voting_keypair: Arc<Keypair>,
```

```
},
/// Web3Signer 서버를 통한 원격 서명
Web3Signer {
signing_url: Url,
http_client: Client,
voting_public_key: PublicKey,
},
}
```

서명 가능한 메시지 타입

Lighthouse는 다양한 타입의 메시지에 대한 서명을 지원합니다:

```
pub  enum SignableMessage<'a, E: EthSpec, Payload: AbstractExecPayload<E> = Full
RandaoReveal(Epoch), // RANDAO 공개값
BeaconBlock(&'a BeaconBlock<E, Payload>), // 비콘 블록
AttestationData(&'a AttestationData), // 증명 데이터
SignedAggregateAndProof(AggregateAndProofRef<'a, E>), // 집계 증명
SelectionProof(Slot), // 선택 증명
SyncSelectionProof(&'a SyncAggregatorSelectionData), // 동기화 선택 증명
SyncCommitteeSignature { // 동기화 위원회 서명
beacon_block_root: Hash256,
slot: Slot,
},
```

```
SignedContributionAndProof(&'a ContributionAndProof<E>), // 기여 증명
ValidatorRegistration(&'a ValidatorRegistrationData), // 검증자 등록
VoluntaryExit(&'a VoluntaryExit), // 자발적 탈퇴
}
```

서명 컨텍스트 (SigningContext)

서명 시 재생 공격을 방지하기 위한 추가 정보를 제공합니다:

```
pub struct SigningContext {
pub domain: Domain, // 도메인 타입
pub epoch: Epoch, // 에포크
pub fork: Fork, // 포크 정보
pub genesis_validators_root: Hash256, // 제네시스 검증자 루트
}

impl SigningContext {
/// 서명에 혼합될 도메인 해시를 반환
pub fn domain_hash(&self, spec: &ChainSpec) -> Hash256 {
spec.get_domain(
self.epoch,
self.domain,
&self.fork,
```

```
self.genesis_validators_root,
)
}
}
```

로컬 키스토어 서명 구현

로컬 키스토어를 사용한 서명은 다음과 같이 구현됩니다:

```
impl SigningMethod {
pub  async  fn  get_signature_from_root<E: EthSpec, Payload: AbstractExecPayload
&self,
signable_message: SignableMessage<'_, E, Payload>,
signing_root: Hash256,
executor: &TaskExecutor,
fork_info: Option<ForkInfo>,
) -> Result<Signature, Error> {
match  self {
SigningMethod::LocalKeystore { voting_keypair, .. } => {
// 메트릭 타이머 시작
let  _timer = validator_metrics::start_timer_vec(
&validator_metrics::SIGNING_TIMES,
&[validator_metrics::LOCAL_KEYSTORE],
);
```

```
let voting_keypair = voting_keypair.clone();
// 블로킹 작업을 별도 스레드에서 실행하여 tokio executor 차단 방지
let signature = executor
.spawn_blocking_handle(
move || voting_keypair.sk.sign(signing_root),
"local_keystore_signer",
)
.ok_or(Error::ShuttingDown)?
.await
.map_err(|e| Error::TokioJoin(e.to_string()))?;
Ok(signature)
}
// Web3Signer 구현...
}
}
}
```

Web3Signer 원격 서명 구현

Web3Signer를 통한 원격 서명은 HTTP API를 사용합니다:

```
SigningMethod::Web3Signer {
signing_url,
```

```
http_client,
..
} => {
// 메시지를 Web3Signer 형식으로 변환
let  object = match  signable_message {
SignableMessage::RandaoReveal(epoch) => {
Web3SignerObject::RandaoReveal { epoch }
}
SignableMessage::BeaconBlock(block) => Web3SignerObject::beacon_block(block)?,
SignableMessage::AttestationData(a) => Web3SignerObject::Attestation(a),
// ... 기타 메시지 타입들
};

let  request = SigningRequest {
message_type: object.message_type(),
fork_info,
signing_root,
object,
};

// HTTP 요청을 통한 서명 요청
let  response: SigningResponse = http_client
.post(signing_url.clone())
```

```
.header(ACCEPT, "application/json")

.json(&request)

.send()

.await

.map_err(|e| Error::Web3SignerRequestFailed(e.to_string()))?

.error_for_status()

.map_err(|e| Error::Web3SignerRequestFailed(e.to_string()))?

.json()

.await

.map_err(|e| Error::Web3SignerJsonParsingFailed(e.to_string()))?;



Ok(response.signature)

}
```

슬래싱 보호 (SlashingProtection) 구현

SlashingDatabase 구조

슬래싱 보호는 SQLite 데이터베이스를 사용하여 구현됩니다:

```
// validator_client/slashing_protection/src/slashing_database.rs

#[derive(Debug, Clone)]

pub  struct SlashingDatabase {

conn_pool: Pool, // SQLite 연결 풀
```

```
}

/// 연결 풀 크기를 1로 설정하여 EXCLUSIVE 잠금 모드와 호환
pub const POOL_SIZE: u32 = 1;
pub const CONNECTION_TIMEOUT: Duration = Duration::from_secs(5);
```

데이터베이스 스키마

슬래싱 보호 데이터베이스는 다음 테이블들로 구성됩니다:

```
-- 검증자 테이블
CREATE TABLE validators (
id INTEGER PRIMARY KEY,
public_key BLOB NOT NULL UNIQUE,
enabled BOOL NOT NULL DEFAULT TRUE
);

-- 서명된 블록 테이블
CREATE TABLE signed_blocks (
validator_id INTEGER NOT NULL,
slot INTEGER NOT NULL,
signing_root BLOB NOT NULL,
FOREIGN KEY(validator_id) REFERENCES validators(id),
UNIQUE (validator_id, slot)
```

```
);


-- 서명된 증명 테이블
CREATE TABLE signed_attestations (
validator_id INTEGER,
source_epoch INTEGER NOT NULL,
target_epoch INTEGER NOT NULL,
signing_root BLOB NOT NULL,
FOREIGN KEY(validator_id) REFERENCES validators(id),
UNIQUE (validator_id, target_epoch)
);
```

블록 제안 슬래싱 검사

블록 제안에 대한 슬래싱 검사는 다음과 같이 구현됩니다:

```
impl SlashingDatabase {
fn check_block_proposal(
&self,
txn: &Transaction,
validator_pubkey: &PublicKeyBytes,
slot: Slot,
signing_root: SigningRoot,
) -> Result<Safe, NotSafe> {
```

```
let validator_id = self.get_validator_id_in_txn(txn, validator_pubkey)?;


// 동일한 슬롯에 이미 서명된 블록이 있는지 확인
let existing_block = txn
.prepare(
"SELECT slot, signing_root
FROM signed_blocks
WHERE validator_id = ?1 AND slot = ?2",
)?
.query_row(params![validator_id, slot], SignedBlock::from_row)
.optional()?;


if let Some(existing_block) = existing_block {
if existing_block.signing_root == signing_root {
// 동일한 슬롯, 동일한 해시 -> 이전에 서명한 블록 재전송
return Ok(Safe::SameData);
} else {
// 동일한 슬롯, 다른 해시 -> 이중 블록 제안 (슬래싱)
return Err(NotSafe::InvalidBlock(InvalidBlock::DoubleBlockProposal(
existing_block,
)));
}
}
```

```
// 최소 슬롯 제한 확인 (하위 경계 위반 방지)
let  min_slot = txn
.prepare("SELECT MIN(slot) FROM signed_blocks WHERE validator_id = ?1")?
.query_row(params![validator_id], |row| row.get(0))?;


if  let Some(min_slot) = min_slot {
if  slot <= min_slot {
return Err(NotSafe::InvalidBlock(
InvalidBlock::SlotViolatesLowerBound {
block_slot: slot,
bound_slot: min_slot,
},
));
}
}


Ok(Safe::Valid)
}
}
```

증명 슬래싱 검사

증명에 대한 슬래싱 검사는 더 복잡한 규칙을 적용합니다:

```
fn check_attestation(
&self,
txn: &Transaction,
validator_pubkey: &PublicKeyBytes,
att_source_epoch: Epoch,
att_target_epoch: Epoch,
att_signing_root: SigningRoot,
) -> Result<Safe, NotSafe> {
// 1. 기본 유효성 검사: source epoch이 target epoch을 초과하지 않는지 확인
if att_source_epoch > att_target_epoch {
return Err(NotSafe::InvalidAttestation(
InvalidAttestation::SourceExceedsTarget,
));
}

let validator_id = self.get_validator_id_in_txn(txn, validator_pubkey)?;

// 2. 이중 투표 검사: 동일한 target epoch에 대한 다른 증명이 있는지 확인
let same_target_att = txn
.prepare(
"SELECT source_epoch, target_epoch, signing_root
FROM signed_attestations
```

```
WHERE validator_id = ?1 AND target_epoch = ?2",
)?
.query_row(
params![validator_id, att_target_epoch],
SignedAttestation::from_row,
)
.optional()?;


if let Some(existing_attestation) = same_target_att {
if existing_attestation.signing_root == att_signing_root {
return Ok(Safe::SameData); // 동일한 증명 재전송
} else {
return Err(NotSafe::InvalidAttestation(InvalidAttestation::DoubleVote(
existing_attestation,
))); // 이중 투표
}
}


// 3. 둘러싸는 투표 검사: 이전 투표가 현재 투표를 둘러싸는지 확인
let surrounding_attestation = txn
.prepare(
"SELECT source_epoch, target_epoch, signing_root
FROM signed_attestations
```

```
WHERE validator_id = ?1 AND source_epoch < ?2 AND target_epoch > ?3
ORDER BY target_epoch DESC
LIMIT 1",
)?
.query_row(
params![validator_id, att_source_epoch, att_target_epoch],
SignedAttestation::from_row,
)
.optional()?;


if  let Some(prev) = surrounding_attestation {
return Err(NotSafe::InvalidAttestation(
InvalidAttestation::PrevSurroundsNew { prev },
));
}


// 4. 둘러싸인 투표 검사: 현재 투표가 이전 투표를 둘러싸는지 확인
let  surrounded_attestation = txn
.prepare(
"SELECT source_epoch, target_epoch, signing_root
FROM signed_attestations
WHERE validator_id = ?1 AND source_epoch > ?2 AND target_epoch < ?3
ORDER BY target_epoch DESC
```

```
LIMIT 1",
)?
.query_row(
params![validator_id, att_source_epoch, att_target_epoch],
SignedAttestation::from_row,
)
.optional()?;

if let Some(prev) = surrounded_attestation {
return Err(NotSafe::InvalidAttestation(
InvalidAttestation::NewSurroundsPrev { prev },
));
}

// 5. 하위 경계 검사
let min_source = txn
.prepare("SELECT MIN(source_epoch) FROM signed_attestations WHERE validator_id =
.query_row(params![validator_id], |row| row.get(0))?;

if let Some(min_source) = min_source {
if att_source_epoch < min_source {
return Err(NotSafe::InvalidAttestation(
InvalidAttestation::SourceLessThanLowerBound {
```

```
source_epoch: att_source_epoch,
bound_epoch: min_source,
},
));
}
}

Ok(Safe::Valid)
}
```

원자적 검사 및 삽입

슬래싱 보호의 핵심은 검사와 삽입이 원자적으로 수행되는 것입니다:

```
pub fn check_and_insert_block_signing_root(
&self,
validator_pubkey: &PublicKeyBytes,
slot: Slot,
signing_root: SigningRoot,
) -> Result<Safe, NotSafe> {
let mut conn = self.conn_pool.get()?;
// EXCLUSIVE 트랜잭션으로 동시성 제어
let txn = conn.transaction_with_behavior(TransactionBehavior::Exclusive)?;
let safe = self.check_and_insert_block_signing_root_txn(
```

```
validator_pubkey,
slot,
signing_root,
&txn,
)?;
txn.commit()?; // 모든 작업이 성공한 경우에만 커밋
Ok(safe)
}


fn check_and_insert_block_signing_root_txn(
&self,
validator_pubkey: &PublicKeyBytes,
slot: Slot,
signing_root: SigningRoot,
txn: &Transaction,
) -> Result<Safe, NotSafe> {
// 먼저 슬래싱 검사 수행
let safe = self.check_block_proposal(txn, validator_pubkey, slot, signing_root)


// 새로운 데이터인 경우에만 데이터베이스에 삽입
if safe != Safe::SameData {
self.insert_block_proposal(txn, validator_pubkey, slot, signing_root)?;
}
```

```
Ok(safe)

}
```

통합 검증자 스토어 구현

LighthouseValidatorStore 구조

검증자 스토어는 서명 방법과 슬래싱 보호를 통합합니다:

```
// validator_client/lighthouse_validator_store/src/lib.rs

pub struct LighthouseValidatorStore<T, E> {

validators: Arc<RwLock<InitializedValidators>>,

slashing_protection: SlashingDatabase,

slashing_protection_last_prune: Arc<Mutex<Epoch>>,

genesis_validators_root: Hash256,

spec: Arc<ChainSpec>,

// ... 기타 필드들

}
```

안전한 블록 서명 구현

블록 서명은 슬래싱 보호와 통합되어 다음과 같이 구현됩니다:

```
async fn sign_abstract_block<Payload: AbstractExecPayload<E>>(

&self,

validator_pubkey: PublicKeyBytes,
```

```
block: BeaconBlock<E, Payload>,

current_slot: Slot,

) -> Result<SignedBeaconBlock<E, Payload>, Error> {

// 1. 슬롯 유효성 검사

if  block.slot() > current_slot {

warn!(

block_slot = block.slot().as_u64(),

current_slot = current_slot.as_u64(),

"Not signing block with slot greater than current slot"

);

return Err(Error::GreaterThanCurrentSlot {

slot: block.slot(),

current_slot,

});

}



// 2. 서명 컨텍스트 생성

let  signing_epoch = block.epoch();

let  signing_context = self.signing_context(Domain::BeaconProposer, signing_epoc

let  domain_hash = signing_context.domain_hash(&self.spec);



// 3. 도플갱어 보호가 적용된 서명 방법 획득

let  signing_method = self.doppelganger_checked_signing_method(validator_pubkey)
```

```
// 4. 슬래싱 보호 검사 (필요한 경우)
let slashing_status = if signing_method
.requires_local_slashing_protection(self.enable_web3signer_slashing_protection)
{
self.slashing_protection.check_and_insert_block_proposal(
&validator_pubkey,
&block.block_header(),
domain_hash,
)
} else {
Ok(Safe::Valid)
};

// 5. 슬래싱 상태에 따른 처리
match slashing_status {
Ok(Safe::Valid) => {
// 안전한 서명 수행
validator_metrics::inc_counter_vec(
&validator_metrics::SIGNED_BLOCKS_TOTAL,
&[validator_metrics::SUCCESS],
);
```

```
let  signature = signing_method
.get_signature(
SignableMessage::BeaconBlock(&block),
signing_context,
&self.spec,
&self.task_executor,
)
.await?;
Ok(SignedBeaconBlock::from_block(block, signature))
}
Ok(Safe::SameData) => {
// 이미 서명된 동일한 데이터
warn!("Skipping signing of previously signed block");
validator_metrics::inc_counter_vec(
&validator_metrics::SIGNED_BLOCKS_TOTAL,
&[validator_metrics::SAME_DATA],
);
Err(Error::SameData)
}
Err(NotSafe::UnregisteredValidator(pk)) => {
// 등록되지 않은 검증자
warn!(
msg = "Carefully consider running with --init-slashing-protection",
```

```
public_key = ?pk,
"Not signing block for unregistered validator"
);
validator_metrics::inc_counter_vec(
&validator_metrics::SIGNED_BLOCKS_TOTAL,
&[validator_metrics::UNREGISTERED],
);
Err(Error::Slashable(NotSafe::UnregisteredValidator(pk)))
}
Err(e) => {
// 슬래싱 위험
crit!(
error = ?e,
"Not signing slashable block"
);
validator_metrics::inc_counter_vec(
&validator_metrics::SIGNED_BLOCKS_TOTAL,
&[validator_metrics::SLASHABLE],
);
Err(Error::Slashable(e))
}
}
```

```
}
```

슬래싱 보호 데이터 교환 (Interchange)

EIP-3076 호환성

Lighthouse는 EIP-3076 슬래싱 보호 교환 형식을 지원합니다:

```
// validator_client/slashing_protection/src/interchange.rs

#[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]

pub struct Interchange {

pub metadata: InterchangeMetadata,

pub data: Vec<InterchangeData>,

}


#[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]

pub struct InterchangeMetadata {

pub interchange_format_version: u64,

pub genesis_validators_root: Hash256,

}


#[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]

pub struct InterchangeData {

pub pubkey: PublicKeyBytes,
```

```
pub signed_blocks: Vec<SignedBlock>,
pub signed_attestations: Vec<SignedAttestation>,
}
```

데이터 가져오기 및 내보내기

```
impl SlashingDatabase {
/// 다른 클라이언트에서 슬래싱 보호 데이터 가져오기
pub fn import_interchange_info(
&self,
interchange: Interchange,
genesis_validators_root: Hash256,
) -> Result<Vec<InterchangeImportOutcome>, InterchangeError> {
// 버전 호환성 검사
let version = interchange.metadata.interchange_format_version;
if version != SUPPORTED_INTERCHANGE_FORMAT_VERSION {
return Err(InterchangeError::UnsupportedVersion(version));
}

// 제네시스 검증자 루트 일치 확인
if genesis_validators_root != interchange.metadata.genesis_validators_root {
return Err(InterchangeError::GenesisValidatorsMismatch {
client: genesis_validators_root,
```

```
interchange_file: interchange.metadata.genesis_validators_root,
});
}

// 원자적 배치 가져오기
let mut conn = self.conn_pool.get()?;
let txn = conn.transaction()?;

let mut import_outcomes = vec![];
let mut commit = true;

for record in interchange.data {
let pubkey = record.pubkey;
match self.import_interchange_record(record, &txn) {
Ok(summary) => {
import_outcomes.push(InterchangeImportOutcome::Success { pubkey, summary });
}
Err(error) => {
import_outcomes.push(InterchangeImportOutcome::Failure { pubkey, error });
commit = false;
}
}
}
```

```
if  commit {
txn.commit()?;
Ok(import_outcomes)
} else {
Err(InterchangeError::AtomicBatchAborted(import_outcomes))
}
}
}
```

보안 고려사항

1. 동시성 제어

슬래싱 보호 데이터베이스는 EXCLUSIVE 잠금 모드를 사용하여 동시 접근을 제어합니다:

```
fn  apply_pragmas(conn: &mut rusqlite::Connection) -> Result<(), rusqlite::Error
conn.pragma_update(None, "foreign_keys", true)?;
// EXCLUSIVE 잠금 모드로 동시 접근 방지
conn.pragma_update(None, "locking_mode", "EXCLUSIVE")?;
Ok(())
}
```

2. 도플갱어 보호

검증자 스토어는 도플갱어 보호를 통해 동일한 검증자가 여러 인스턴스에서 실행되는 것을 방지합니다:

```
fn doppelganger_checked_signing_method(
&self,
validator_pubkey: PublicKeyBytes,
) -> Result<Arc<SigningMethod>, Error> {
if self.doppelganger_protection_allows_signing(validator_pubkey) {
self.validators
.read()
.signing_method(&validator_pubkey)
.ok_or(Error::UnknownPubkey(validator_pubkey))
} else {
Err(Error::DoppelgangerProtected(validator_pubkey))
}
}
```

3. 키스토어 잠금

로컬 키스토어는 파일 시스템 수준에서 잠금을 사용하여 동시 접근을 방지합니다:

```
SigningMethod::LocalKeystore {
voting_keystore_lockfile: Mutex<Option<Lockfile>>,
// ... 기타 필드들
```

```
}
```

성능 최적화

1. 연결 풀링

SQLite 연결 풀을 사용하여 데이터베이스 연결을 효율적으로 관리합니다:

```
pub const POOL_SIZE: u32 = 1; // EXCLUSIVE 모드와 호환
pub const CONNECTION_TIMEOUT: Duration = Duration::from_secs(5);

fn open_conn_pool(path: &Path) -> Result<Pool, NotSafe> {
let manager = SqliteConnectionManager::file(path)
.with_flags(rusqlite::OpenFlags::SQLITE_OPEN_READ_WRITE)
.with_init(Self::apply_pragmas);
let conn_pool = Pool::builder()
.max_size(POOL_SIZE)
.connection_timeout(CONNECTION_TIMEOUT)
.build(manager)?;
Ok(conn_pool)
}
```

2. 비동기 서명

로컬 키스토어 서명은 별도의 블로킹 스레드에서 실행되어 메인 tokio executor를 차단하지 않습니다:

```
let signature = executor
.spawn_blocking_handle(
move || voting_keypair.sk.sign(signing_root),
"local_keystore_signer",
)
.ok_or(Error::ShuttingDown)?
.await?;
```

3. 메트릭 수집

서명 성능과 슬래싱 보호 상태를 모니터링하기 위한 메트릭을 수집합니다:

```
validator_metrics::inc_counter_vec(
&validator_metrics::SIGNED_BLOCKS_TOTAL,
&[validator_metrics::SUCCESS],
);
```

```
let _timer = validator_metrics::start_timer_vec(
&validator_metrics::SIGNING_TIMES,
&[validator_metrics::LOCAL_KEYSTORE],
);
```

결론

Lighthouse의 서명 및 슬래싱 보호 구현은 다음과 같은 핵심 특징을 가집니다:

1. **다중 서명 방법 지원**: 로컬 키스토어와 Web3Signer 원격 서명을 모두 지원
2. **강력한 슬래싱 보호**: SQLite 기반의 원자적 검사 및 삽입으로 슬래싱 방지
3. **EIP-3076 호환성**: 표준 교환 형식을 통한 클라이언트 간 데이터 이동 지원
4. **보안 우선 설계**: 도플갱어 보호, 파일 잠금, 동시성 제어 등 다층 보안
5. **성능 최적화**: 비동기 처리, 연결 풀링, 메트릭 수집을 통한 효율적 운영

이러한 구현을 통해 Lighthouse는 검증자가 안전하고 효율적으로 Ethereum 2.0 네트워크에 참여할 수 있도록 지원합니다.